SIMATS ENGINEERING

**NAME: S.SUSITHRA**
**REG NO.: 1912260033**
**COURSE NAME: ARTIFICIAL INTELLIGENCE**
**COURSE CODE: CSA1710**

# HOSPITAL MEDICINE-DELIVERY ROBOT

AI Search Algorithms, Complexity Analysis, Experimental Evaluation and Day-4 Localisation Challenge

Assignment report covering Tasks 1–12

## TASK 1 – DESIGN THE AGENT AND THE SEARCH ALGORITHMS

### 1.1 PEAS Description

| Component | Description |
|---|---|
| Performance | Deliver medicines correctly, minimize travel time, avoid collisions and reach wards safely. |
| Environment | Hospital corridors, junctions, lifts, pharmacy and wards. |
| Actuators | Wheels, motors, steering, brakes and lift-control interface. |
| Sensors | Camera, LiDAR, ultrasonic sensors, location sensors and obstacle detectors. |

### 1.2 Environment Classification

| Property | Classification | Justification |
|---|---|---|
| Observability | Fully observable (original problem) | The robot knows its current location and hospital map. |
| Determinism | Deterministic | Each action has a known destination and travel cost. |
| Episodic / Sequential | Sequential | Every movement affects the next movement. |
| Static / Dynamic | Dynamic | People, obstacles and lift availability may change. |

| | | |
|---|---|---|
| Discrete / Continuous | Primarily discrete | Locations and actions are represented as graph nodes and edges, although physical movement is continuous. |
| Agents | Single-agent | One robot makes the route-planning decisions. |

## 1.3 AI Approach

The robot follows the acting rationally approach. A rational agent selects actions that maximize its performance based on available information. The robot selects a route that safely delivers medicines while minimizing travel time.

This problem is related to classical AI problem-solving approaches developed during the 1950s and 1960s. Early AI systems represented problems as states and searched for sequences of actions leading to a goal. The hospital robot applies the same state-space search concept.

## 1.4 Problem Formulation

| Element | Description |
|---|---|
| State | Current location of the robot. |
| Initial state | Central pharmacy. |
| Goal state | Requested ward. |
| Actions | Move to any directly connected location. |
| Transition model | Moving from one node to an adjacent node. |
| Goal test | Check whether the current location is the requested ward. |
| Path cost | Total travel time of all movements. |

Represent the hospital as G = (V, E), where V is the set of hospital locations, E is the set of connections between locations, and c(u, v) is the travel-time cost between locations.

## 1.5 Proposed Uninformed Search Algorithms

Three different approaches are selected: Breadth-First Search (BFS), Uniform-Cost Search (UCS), and recursive Depth-First Search (DFS). They differ in their node-selection strategies and memory requirements.

### Algorithm 1: Breadth-First Search (BFS)

Basic idea: BFS explores the hospital level by level. It visits all nearby locations before exploring locations farther away. Data structure: FIFO queue.

```
BFS(Graph, Start, Goal)
1. Create an empty queue.
2. Insert Start into the queue.
3. Mark Start as visited.
4. While queue is not empty:
   a. Remove the front node.
   b. If node is Goal, return the path.
   c. For every unvisited neighbour:
      - Mark it visited.
      - Store its parent.
      - Insert it into the queue.
5. Return FAILURE.
```

Justification: BFS is complete for finite graphs and finds a route with the minimum number of edges. It provides a useful baseline for comparing other algorithms. It does not necessarily find the minimum travel-time route when edge costs differ.

#### Algorithm 2: Uniform-Cost Search (UCS)

Basic idea: UCS expands the location having the lowest accumulated travel cost. Data structure: priority queue (min-heap).

```
UCS(Graph, Start, Goal)
1. Insert Start with cost 0 into priority queue.
2. Set best cost of Start = 0.
3. While priority queue is not empty:
   a. Remove node with minimum cost.
   b. If node is Goal, return the path.
   c. For every neighbour:
      - Calculate new cost.
      - If new cost is smaller:
        * Update best cost.
        * Store parent.
        * Insert neighbour into queue.
4. Return FAILURE.
```

Justification: Hospital corridors have different travel times. UCS considers these costs and finds the minimum-cost route when all edge costs are non-negative.

#### Algorithm 3: Recursive Depth-First Search (DFS)

Basic idea: DFS follows one path as deeply as possible. When it reaches a dead end, it backtracks and explores another path. Data structure: recursion stack and explored set.

```
DFS(Graph, Current, Goal, Visited)
1. Mark Current as visited.
2. If Current = Goal, return SUCCESS.
3. For each neighbour of Current:
   If neighbour is not visited:
      Store parent.
      Call DFS recursively.
      If SUCCESS, return SUCCESS.
4. Return FAILURE.
```

Justification: DFS investigates recursive problem-solving and has low search-frontier memory requirements. It is useful when a solution may be located deep in the hospital graph. However, DFS does not guarantee the shortest or least-cost route.

### 1.6 Preprocessing Requirements

- Represent hospital locations using node IDs.
- Construct an adjacency list.
- Store travel-time costs for every connection.
- Create a visited set for duplicate detection.
- Initialize parent and cost dictionaries.

Let V be the number of vertices and E the number of edges. Adjacency-list construction takes O(V + E) time and space. Visited-set initialization takes O(V) time and space. Parent and cost initialization can take up to O(V). Total preprocessing is T□ = O(V + E) and S□ = O(V + E). For a sparse hospital graph, E = O(V), so preprocessing is approximately linear.

## TASK 2 – IDENTIFY THE BASIC OPERATION

| Algorithm | Basic operation | Where it occurs | Reason |
|---|---|---|---|

| | | | |
|---|---|---|---|
| BFS | Node expansion and successor generation | When removing a node and examining its neighbours. | Each expanded node generates its connected neighbours. |
| UCS | Priority-queue insertion and minimum-cost extraction | During cost updates and node selection. | The algorithm repeatedly selects and updates the lowest-cost node. |
| Recursive DFS | Recursive calls and successor examination | During recursive exploration of neighbours. | Each recursive call investigates a deeper location. |

In BFS, increasing the number of locations increases the number of nodes expanded and neighbours examined. In UCS, additional locations create more priority-queue operations. In recursive DFS, recursive calls increase as search depth and branching factor increase.

## TASK 3 – DERIVE TIME AND SPACE COMPLEXITY MATHEMATICALLY

### 3.1 Input Parameters

| Symbol | Meaning |
|---|---|
| b | Branching factor. |
| d | Depth of the shallowest goal. |
| m | Maximum search depth. |
| C* | Optimal solution cost. |
| ε | Minimum positive step cost. |

### 3.2 BFS Complexity Derivation

BFS explores nodes level by level. The number of nodes at levels 0 through d is 1, b, $b^2$, …, $b^d$.

Total nodes generated: $N = 1 + b + b^2 + \dots + b^d$.

Using the geometric series: $N = (b^{(d+1)} - 1)/(b - 1)$. Therefore, tree-search time is $\Theta(b^d)$, and the frontier space is $\Theta(b^d)$.

For graph search with duplicate detection, tighter finite-graph bounds are $T_{BFS} = O(V + E)$ and $S_{BFS} = O(V)$.

### 3.3 UCS Complexity Derivation

UCS expands nodes in increasing order of path cost. Since every step costs at least ε, a path with cost at most C* can have at most $L = \text{floor}(C^*/\varepsilon)$ steps.

The number of nodes up to this depth is $N = 1 + b + b^2 + \dots + b^L = (b^{(L+1)} - 1)/(b - 1)$. Thus, the cost-bounded search-tree expansion is $O(b^{(1 + \text{floor}(C^*/\varepsilon))})$.

For a binary-heap priority queue, insertion and extraction take O(log V). With suitable duplicate handling on a finite graph with non-negative costs, $T_{UCS} = O((V + E) \log V)$, and $S_{UCS} = O(V + E)$. The cost-bounded tree bound and finite-graph bound describe different models.

### 3.4 Recursive DFS Complexity Derivation

DFS can explore nodes up to maximum depth m. The total number of nodes in a full search tree is $N = 1 + b + b^2 + \dots + b^m = (b^{(m+1)} - 1)/(b - 1)$. Therefore, worst-case tree-search time is $O(b^m)$.

The recursion stack stores the current path and pending neighbour information. For a branching search tree, space is O(bm). If only the current path is counted, stack depth is O(m). A graph-search implementation additionally requires an explored set of up to O(V).

### 3.5 Overall Cost Including Preprocessing

| Algorithm | Search time | Preprocessing time | Total space |
|---|---|---|---|
| BFS | O(V + E) | O(V + E) | O(V + E) |
| UCS | O((V + E) log V) | O(V + E) | O(V + E) |
| DFS | $O(b^m)$ tree-search bound | O(V + E) | O(V + E) with explored set |

Preprocessing is important because a larger hospital requires more memory and time to construct its graph before searching begins. For sparse graphs, adjacency-list construction is approximately O(V).

## TASK 4 – BEST, AVERAGE AND WORST-CASE BEHAVIOUR

### 4.1 BFS

Best case: The goal is the starting location. Only one node is checked, so the search takes O(1) time.

Average case: If approximately half the nodes at depth d are generated before finding the goal, N_avg ≈ 1 + b + … + b^(d−1) + $b^d/2$, giving $\Theta(b^d)$.

Worst case: The goal is the last node examined at depth d, or it is unreachable. N_worst = 1 + b + … + $b^d$, giving $O(b^d)$ tree-search time.

### 4.2 UCS

Best case: The goal is the starting location or is removed from the priority queue very early as the cheapest available goal.

Average case: UCS expands locations whose accumulated costs are below the optimal solution cost, with the exact count depending on the cost distribution and ties. The cost-bounded tree-search estimate is O(b^(1 + C*/ε)).

Worst case: Many low-cost detours exist before the goal is selected. UCS may explore many locations before reaching the goal, especially when C*/ε is large.

### 4.3 Recursive DFS

Best case: The goal is found along the first branch explored. The search follows approximately d nodes, giving O(d).

Average case: The number of recursive calls depends on the goal position and neighbour order. In a balanced tree with a randomly positioned goal at depth d, the expected search may involve roughly half the nodes at that depth plus preceding levels.

Worst case: The goal is in the last branch at maximum depth, or is unreachable. The tree-search bound is $O(b^m)$. The recursion stack may grow to depth m, creating stack-overflow risks.

| Algorithm | Best case | Average case | Worst case |
|---|---|---|---|
| BFS | O(1) | $\Theta(b^d)$ | $O(b^d)$ |
| UCS | Early cheapest goal | Depends on cost distribution | O(b^(1 + C*/ε)) search-tree bound |
| DFS | O(d) | Depends on branch order | $O(b^m)$ |

Actual performance depends on graph structure, edge costs, goal position and the number of duplicate locations.

## TASK 5 – ASYMPTOTIC ANALYSIS

Asymptotic analysis describes how algorithm performance changes as input size increases.

| Algorithm | Time complexity | Space complexity |
|---|---|---|
| BFS | O(V + E) for graph search | O(V) |
| UCS | O((V + E) log V) | O(V + E) |
| DFS | $O(b^m)$ worst-case tree search; O(V + E) finite graph search with duplicate detection | O(V + m) with explored set |

For a sparse graph where E = O(V), BFS is O(V), UCS is O(V log V), and DFS has an exponential worst-case tree-search bound, while finite graph search with duplicate detection is O(V + E).

As the hospital grows, BFS is practical for sparse graphs but its frontier can consume considerable memory. UCS requires additional processing for priority-queue operations. DFS uses less search-frontier memory but can spend considerable time exploring an unhelpful branch.

When the hospital reaches one million locations, memory may become a major limitation for BFS and UCS. DFS may use less frontier memory, but its search time and recursion depth can become problematic. UCS is preferable when travel time is important, while BFS is useful when every movement has equal cost.

## TASK 6 – FORMULATE THE RECURRENCE RELATION

The recursive algorithm selected is DFS. Let T(m) be execution time for DFS with remaining depth m, b the branching factor, T(m−1) the time for one recursive subproblem, and c the constant work for checking the goal and examining a node.

At each node, DFS performs constant work and may recursively explore up to b neighbours. Therefore: T(m) = bT(m−1) + c.

Base case: $T(0) = c_0$ , where $c_0$ is the constant time required to check the current node.

- Original problem size: m.
- Recursive subproblem size: m − 1.
- Number of recursive calls: At most b.
- Work outside recursive calls: c.
- Base case: $T(0) = c_0$ .

This recurrence describes worst-case tree-search behaviour when each branch may need to be explored. Duplicate detection can reduce actual work on a graph.

## TASK 7 – SOLVE THE RECURRENCE RELATION

Use the iteration method to solve T(m) = bT(m−1) + c.

First expansion: T(m) = bT(m−1) + c. Since T(m−1) = bT(m−2) + c, $T(m) = b^2T(m-2) + bc + c$.

Expanding again gives $T(m) = b^3T(m-3) + b^2c + bc + c$.

After m expansions: $T(m) = b^mT(0) + c(1 + b + b^2 + \ldots + b^{\wedge}(m-1))$.

Using the geometric series: $T(m) = b^mc_0 + c((b^m - 1)/(b - 1))$. For b > 1, $T(m) = \Theta(b^m)$.

Interpretation: Recursive DFS execution time increases exponentially in the worst case when branching factor or search depth increases.

### Extension to Iterative Deepening DFS

Iterative Deepening Search repeatedly performs depth-limited DFS with increasing limits. Its total work is $1 + 2b + 3b^2 + \dots + (d+1)b^d$. For $b > 1$, this is $O(b^d)$. Although earlier levels are searched repeatedly, most nodes are at the deepest level, so the repeated work does not change the asymptotic complexity. IDS uses much less memory than BFS while maintaining completeness for finite branching factors and finding the shallowest solution.

## TASK 8 – IMPLEMENT AND TEST THE ALGORITHMS

The following Python program implements BFS, UCS and recursive DFS. The experiment uses a generated hospital graph with a binary-tree-like corridor structure, different travel-time costs and an isolated location for unreachable-goal testing. Graph sizes are 1,000, 10,000, 100,000 and 1,000,000 locations.

```
import heapq
import time
from collections import deque

def create_graph(n):
    graph = [[] for _ in range(n)]
    for child in range(1, n - 1):
        parent = (child - 1) // 2
        cost = 1 + (child % 3)
        graph[parent].append((child, cost))
        graph[child].append((parent, cost))
    return graph

def get_path(parent, goal):
    path = []
    while goal is not None:
        path.append(goal)
        goal = parent[goal]
    return path[::-1]

def bfs(graph, start, goal):
    q = deque([start])
    visited = {start}
    parent = {start: None}
    generated, expanded, max_frontier = 1, 0, 1
    while q:
        u = q.popleft()
        expanded += 1
        if u == goal:
            return get_path(parent, goal), generated, expanded, max_frontier
        for v, cost in graph[u]:
            if v not in visited:
                visited.add(v)
                parent[v] = u
                q.append(v)
                generated += 1
        max_frontier = max(max_frontier, len(q))
    return None, generated, expanded, max_frontier

def ucs(graph, start, goal):
    pq = [(0, start)]
    best = {start: 0}
    parent = {start: None}
    generated, expanded, max_frontier = 1, 0, 1
    while pq:
        cost, u = heapq.heappop(pq)
```

```
        if cost != best[u]:
            continue
        expanded += 1
        if u == goal:
            return get_path(parent, goal), generated, expanded, max_frontier
        for v, weight in graph[u]:
            new_cost = cost + weight
            if new_cost < best.get(v, float("inf")):
                best[v] = new_cost
                parent[v] = u
                heapq.heappush(pq, (new_cost, v))
                generated += 1
        max_frontier = max(max_frontier, len(pq))
    return None, generated, expanded, max_frontier

def recursive_dfs(graph, start, goal):
    visited = set()
    parent = {start: None}
    generated, expanded, max_depth = 1, 0, 0
    def search(u, depth):
        nonlocal generated, expanded, max_depth
        visited.add(u)
        expanded += 1
        max_depth = max(max_depth, depth)
        if u == goal:
            return True
        for v, cost in graph[u]:
            if v not in visited:
                parent[v] = u
                generated += 1
                if search(v, depth + 1):
                    return True
        return False
    if search(start, 0):
        return get_path(parent, goal), generated, expanded, max_depth
    return None, generated, expanded, max_depth

def benchmark(graph, start, goal, algorithm, runs=5):
    times, result = [], None
    for _ in range(runs):
        t1 = time.perf_counter()
        result = algorithm(graph, start, goal)
        t2 = time.perf_counter()
        times.append(t2 - t1)
    return result, sum(times) / runs

sizes = [1000, 10000, 100000, 1000000]
for n in sizes:
    print("\nHospital states:", n)
    graph = create_graph(n)
    start, goal = 0, n - 2
    for name, algorithm in [
        ("BFS", bfs), ("UCS", ucs), ("DFS", recursive_dfs)
    ]:
        result, avg_time = benchmark(graph, start, goal, algorithm)
        path, generated, expanded, frontier = result
        depth = len(path) - 1 if path else None
        path_cost = None
        if path:
            path_cost = sum(
                dict(graph[u]).get(v, 0)
                for u, v in zip(path, path[1:])
            )
        print(name, "Generated:", generated,
```

```
"Expanded:", expanded,
"Frontier/stack proxy:", frontier,
"Depth:", depth, "Cost:", path_cost,
"Average time:", round(avg_time, 6))
```

### 8.1 Test Cases

| Test case | Start | Goal | Expected result |
|---|---|---|---|
| Reachable goal | 0 | N−2 | Route found. |
| Unreachable goal | 0 | N−1 | Failure. |
| Start equals goal | 0 | 0 | Route [0], cost 0. |

### 8.2 Expected Behaviour

- BFS finds the path with the minimum number of edges.
- UCS finds the path with the minimum total travel cost.
- DFS finds a path according to neighbour exploration order.
- All three return failure when the goal is unreachable.
- All three return immediately when the start and goal are identical.

Important implementation note: The benchmark code measures search time separately from graph construction. Report preprocessing time separately as well. The program uses five runs and calculates their average. Actual timings depend on the computer, Python version and available memory, so they must be taken from the executed program rather than invented.

## TASK 9 – RECORD EXPERIMENTAL PERFORMANCE

### 9.1 Performance Table

| States | Algorithm | Nodes generated | Nodes expanded | Maximum frontier / stack |
|---|---|---|---|---|
| 1,000 | BFS | Record output | Record output | Record output |
| 1,000 | UCS | Record output | Record output | Record output |
| 1,000 | DFS | Record output | Record output | Record output |
| 10,000 | BFS | Record output | Record output | Record output |
| 10,000 | UCS | Record output | Record output | Record output |
| 10,000 | DFS | Record output | Record output | Record output |
| 100,000 | BFS | Record output | Record output | Record output |
| 100,000 | UCS | Record output | Record output | Record output |
| 100,000 | DFS | Record output | Record output | Record output |
| 1,000,000 | BFS | Record output | Record output | Record output |
| 1,000,000 | UCS | Record output | Record output | Record output |
| 1,000,000 | DFS | Record output | Record output | Record output |

### 9.2 Additional Performance Measurements

- Average execution time over five runs.

- Solution depth.
- Total travel cost.
- Whether the solution is optimal.
- Maximum frontier size or recursion-stack depth.
- Graph preprocessing time.

### 9.3 Graphs to Prepare

- Graph 1: Number of states vs average execution time.
- Graph 2: Number of states vs nodes expanded.
- Graph 3: Number of states vs maximum frontier size.

Use a logarithmic scale for the Y-axis when values vary greatly.

## TASK 10 – COMPARE THEORY WITH EXPERIMENTAL BEHAVIOUR

| Algorithm | Theoretical behaviour | Expected experimental behaviour |
|---|---|---|
| BFS | Linear in V + E with duplicate detection. | Predictable growth, but large frontier. |
| UCS | O((V + E) log V). | More processing because of priority-queue operations. |
| DFS | Exponential tree-search worst case. | Can perform well or poorly depending on branch order; graph-search duplicate detection limits repeated expansions. |

### 10.1 Reasons for Differences

- Graph structure: Different hospital layouts have different branching factors and depths.
- Duplicate states: Multiple routes may lead to the same junction.
- Tie-breaking: The order in which equal-cost nodes are selected affects exploration.
- Language overhead: Python data structures and recursive function calls require additional processing.
- Hardware: CPU speed and available memory affect execution time.

As the graph grows from 1,000 to 1,000,000 states, BFS and UCS can process the graph efficiently in terms of finite-graph asymptotic bounds, provided duplicate detection is used. UCS may take longer because it maintains a priority queue. DFS can be efficient when the goal is located along an early branch, but inefficient when the goal is located in a late branch. Experimental results should be used to confirm these observations.

## TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

Recommended algorithm: Uniform-Cost Search.

UCS is recommended for the original hospital medicine-delivery robot because hospital corridors have different travel-time costs and UCS considers the accumulated cost of every route. It finds an optimal route when travel costs are non-negative and uses a priority queue with duplicate detection.

| Factor | BFS | UCS | DFS |
|---|---|---|---|
| Minimum-edge path | Yes | Not necessarily | No |
| Minimum-cost path | Only with equal edge costs | Yes | No |

| | | | |
|---|---|---|---|
| Complete | Yes, finite graph | Yes, positive costs | Yes, finite graph with duplicate detection |
| Memory usage | High | High | Explored set plus recursion stack |
| Suitable for weighted routes | Limited | Highly suitable | Limited |

### 11.1 Limitations

- Priority-queue operations increase processing time.
- Large graphs require additional memory.
- Graph updates may require refreshing the stored map.
- A recursive implementation of DFS may exceed Python's recursion limit.

Final recommendation: UCS provides the best balance between route quality and correctness for the original requirement. Although BFS may use less processing time and DFS may require less frontier memory, neither guarantees the minimum travel-time route. For one million or more locations, UCS should use an efficient adjacency list, a binary heap, duplicate detection and updated travel-cost information.

# DAY-4 DESIGN CHALLENGE

## TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

Changed requirement: After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of k possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

### A. Impact Analysis (2 Marks)

In the original problem, the robot knows its exact current location. After the sensor failure, it only knows that it is at one of k possible junctions. This changes the problem from a fully observable environment to a partially observable environment.

- The exact initial state is no longer known.
- The robot maintains multiple possible locations.
- The robot must consider all possible locations before selecting an action.
- Sensor information becomes important for decision-making.

Previously, the state was represented by one location, for example S = {$J_1$ }. Now, the robot maintains a belief state B = {$J_1$ , $J_2$ , …, J□}. A belief state represents all locations where the robot may currently exist. Therefore, the robot's knowledge becomes part of its state representation.

### B. Evaluation of the Original Design (2 Marks)

The original BFS, UCS and DFS algorithms can still be used, but their state representation must be modified. Instead of searching individual locations, the robot searches sets of possible locations.

For N hospital locations, the total number of possible belief states is $2^N$. If exactly k locations are possible, the number of belief states is C(N, k) = N! / (k!(N−k)!). This can become extremely large as the hospital grows.

Therefore, the original design is not directly suitable for uncertain starting locations.

### C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original design. The robot will use Belief-State Search with Local Sensing.

- Identify all possible current locations.
- Store these locations in a belief-state set.
- Use the local sensor to identify the actual junction whenever possible.
- Update the belief state using the sensor observation.
- Calculate a safe route from the updated belief state to the pharmacy.
- Continue until the robot reaches the pharmacy.

Belief-state transition: $B_{t+1} = \bigcup_{s \in B_t} T(s, a)$, where $B_t$ is the current belief state, a is the selected action and T(s, a) is a possible next location after action a.

When a sensor observation o is received, update the belief state to B′ = {s in B : O(s) = o}. The robot removes locations inconsistent with the observation.

Without sensing, search may require exploring up to $2^N$ belief states. With accurate local sensing, the belief state can be reduced to a single location, $B = \{J_i\}$. This reduces uncertainty and makes route planning easier.

## D. Evidence-Based Defence (2 Marks)

Consider a simple hospital corridor. The pharmacy is at position 0. The robot may be at positions 1 to k. Each movement toward the pharmacy takes one step. The local sensor can identify the exact current position.

Without a sensor, the robot maintains all possible locations and repeatedly moves towards the pharmacy until every possible location reaches position 0. In this model, worst-case plan length L = k and the number of belief states generated is k + 1.

With local sensing, the robot identifies its exact location and follows a location-specific route. If starting positions are equally likely, average movement length is $L_{avg} = (1 + 2 + \ldots + k)/k = (k + 1)/2$. The worst-case movement length remains k.

## 12.1 Python Experiment

```
def without_sensor(k):
    belief = set(range(1, k + 1))
    generated = 1
    steps = 0
    while belief != {0}:
        belief = {max(0, x - 1) for x in belief}
        steps += 1
        generated += 1
    return generated, steps

def with_sensor(k):
    generated = 1 + k + 1
    plan_lengths = list(range(1, k + 1))
    average_length = sum(plan_lengths) / k
    worst_length = max(plan_lengths)
    return generated, average_length, worst_length

print("k | Without Sensor | With Sensor")
print("-----------------------------------------")
for k in [2, 4, 8]:
    b1, l1 = without_sensor(k)
    b2, avg, worst = with_sensor(k)
    print(k, "|", b1, "states, plan:", l1,
          "|", b2, "states, average plan:", avg,
          "worst:", worst)
```

## 12.2 Expected Experimental Output

For the specified line-corridor model; plan length counts movements, not the sensing operation.

| Possible locations (k) | Without sensor: belief states | Without sensor: plan length | With sensor: belief states | With sensor: average plan | With sensor: worst plan |
|---|---|---|---|---|---|
| 2 | 3 | 2 | 4 | 1.5 | 2 |
| 4 | 5 | 4 | 6 | 2.5 | 4 |
| 8 | 9 | 8 | 10 | 4.5 | 8 |

## 12.3 Interpretation

- Without a sensor, the robot must select a common action that works for all possible locations.
- With a sensor, it can select a route based on its actual location.
- The sensor reduces uncertainty and the average movement required.
- The worst-case movement remains unchanged in this particular corridor model.

## 12.4 Final Defence

The original UCS algorithm is retained as the underlying route-cost algorithm, but its input is modified to use belief states and sensor observations. This is more appropriate than completely replacing the original design because UCS already provides minimum-cost routes for known locations.

The modified system combines belief-state representation, local sensor observations, belief-state updates and UCS for minimum-cost route selection.

Final decision: Modify the original design using belief-state search and local sensing. This provides a safer and more reliable solution when the robot's localisation sensor fails.